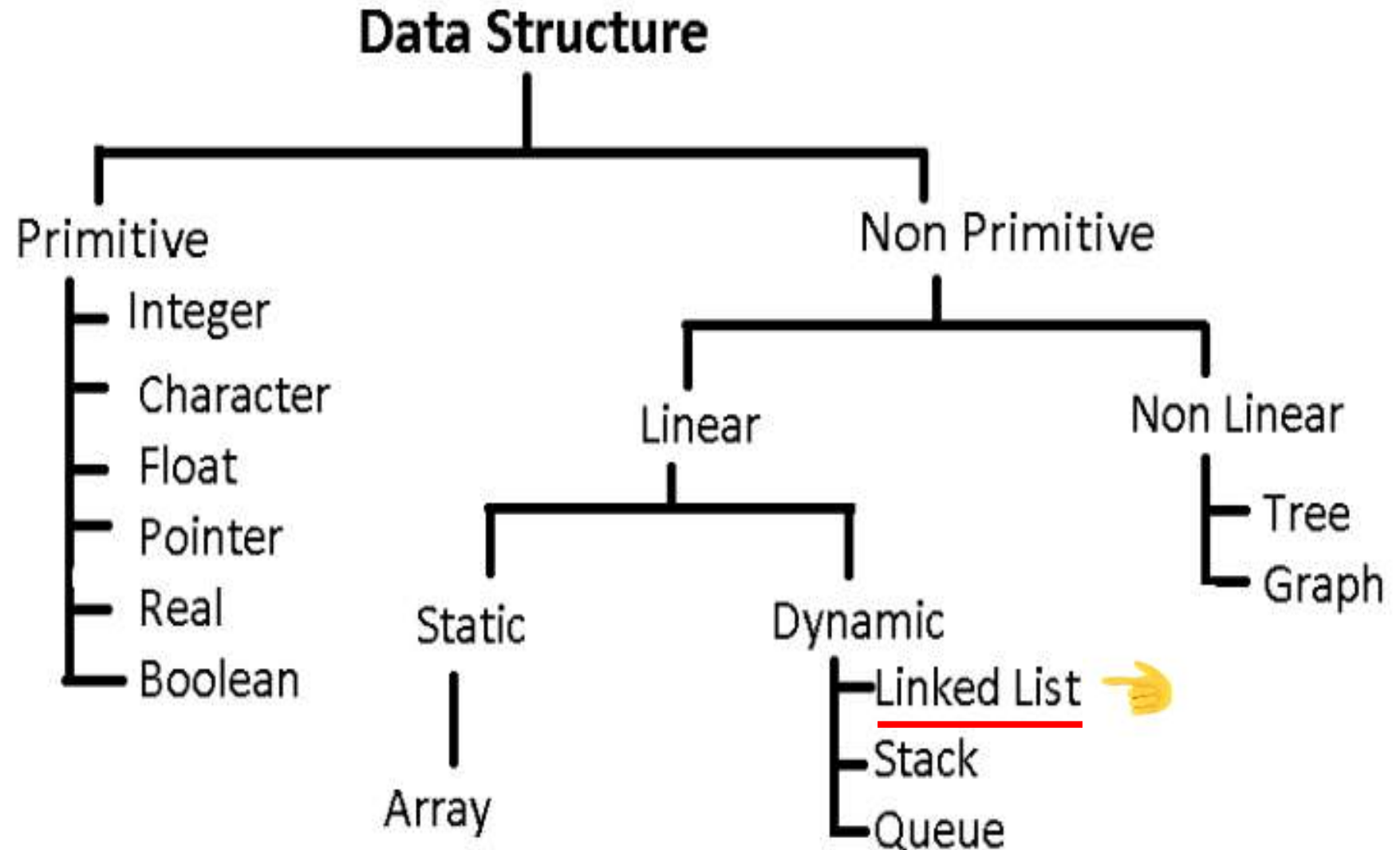


LINKED LISTS

(SINGLE & DOUBLE)

Dr. Jyothi Pillai
Professor
Bhilai Institute of Technology Durg(C.G.)

LINKED LIST



Why Linked Lists?

Limitations Of Arrays

- ❖ Arrays are simple to understand and elements of an array are easily accessible, but arrays have some limitations.
- ❖ Array elements are always stored in contiguous memory locations.
- ❖ Static memory allocation—Memory allotment is required at the compile time.
- ❖ Wastage of space—inefficient usage of storage memory.
- ❖ Prediction of size – Size of an array must be specified precisely at the beginning which may be difficult task in many practical applications.
- ❖ Once size of array is defined, it can not be increased or decreased during execution.
- ❖ Operations of array like insertions or deletions are pretty tedious as they require shift operations.

To over come these limitations, linked list can be used.

Why Linked Lists?

Advantages of Linked Lists

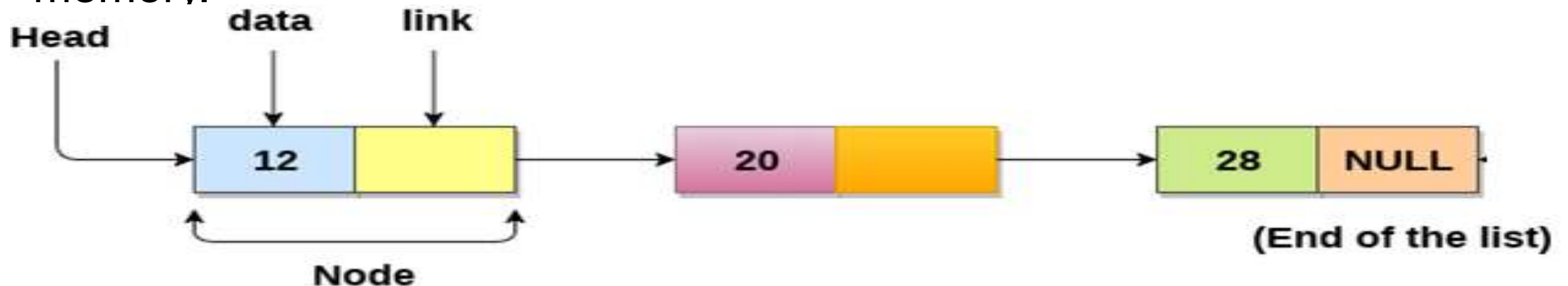
- ❖ Linked Lists are dynamic in nature which allocates the memory space dynamically when required.
- ❖ The unused space can be released in the situation where allocated space seems to be more.
- ❖ Insertion and deletion operations can be easily implemented and are less time consuming.
- ❖ Size of linked list can grow or shrink in size during the execution of a program.
- ❖ In Linked Lists, no need to know the size in advance
- ❖ Stacks and queues can be easily executed.
- ❖ Linked List provides flexibility in allowing the items to be rearranged efficiently.

Linked Lists

- A linked list is a linear data structure that contains a sequence of elements called **nodes** such that each node links to its next node in the sequence.
- Nodes are randomly stored in the memory.
- Each node has two f

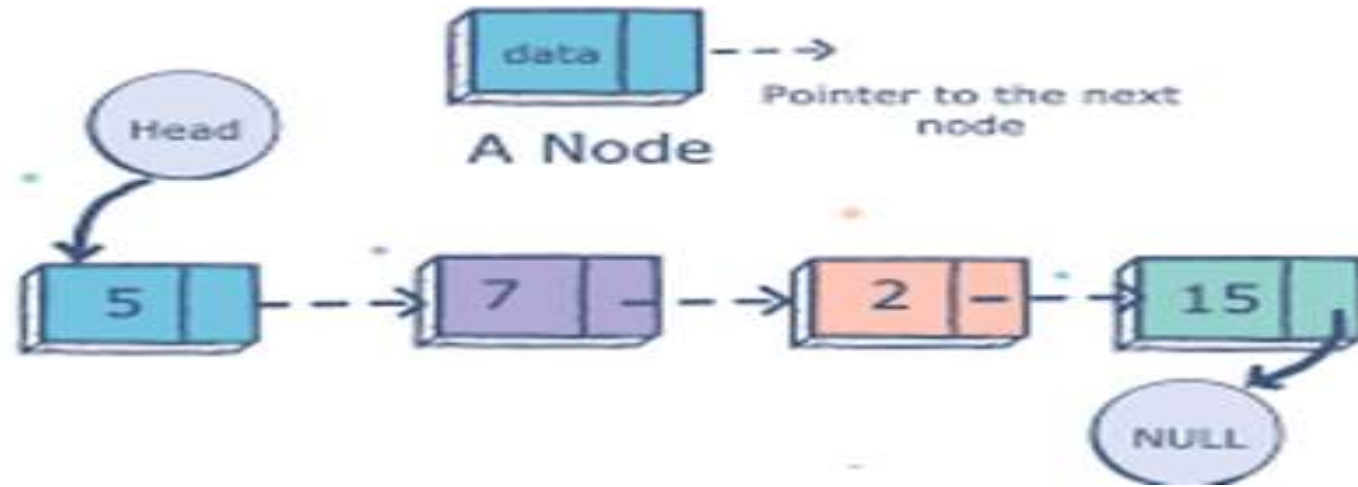


- **Data** field stores actual data stored at that particular address.
- **Link** contains address of next node to point to the next node in the memory.

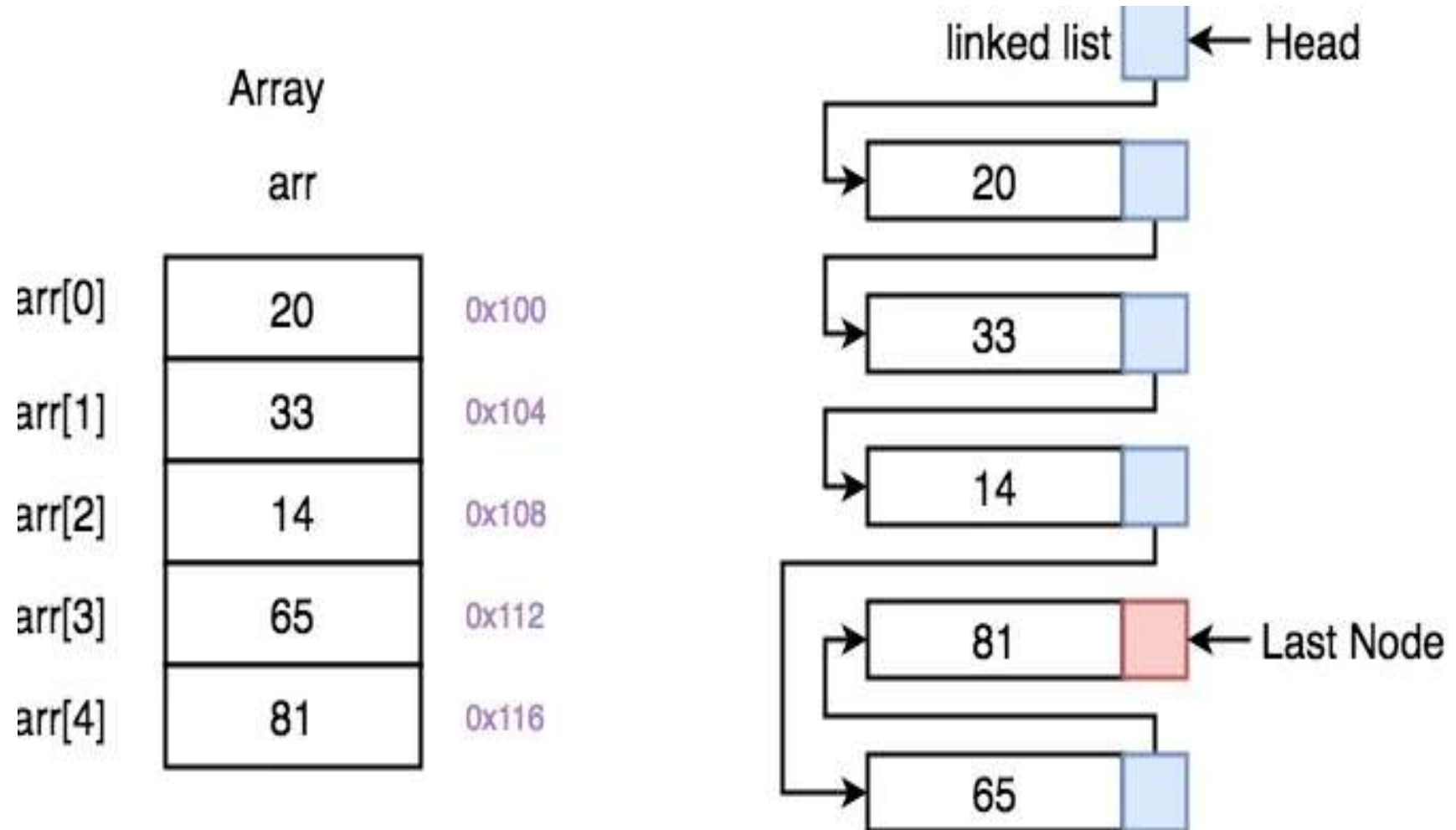


Linked Lists

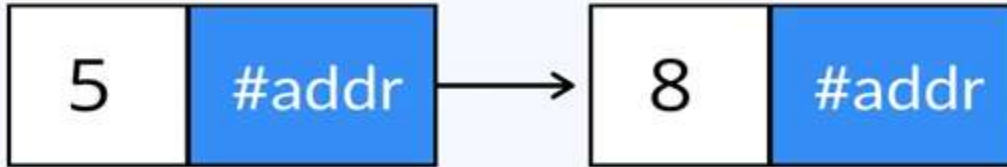
- Linked Lists are used to create trees and graphs.



Representation of Array and Linked List



Types of Linked Lists

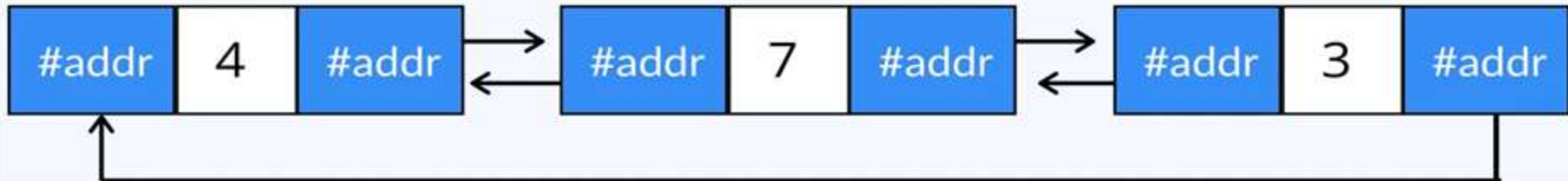


Singly Linked List



Doubly Linked List

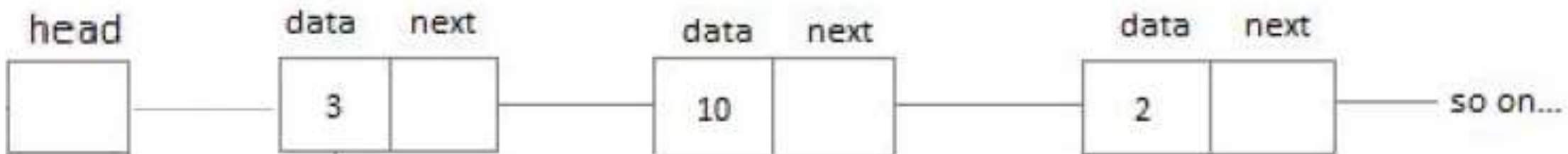
Circular Linked List



Types of lists

Singly Linked list

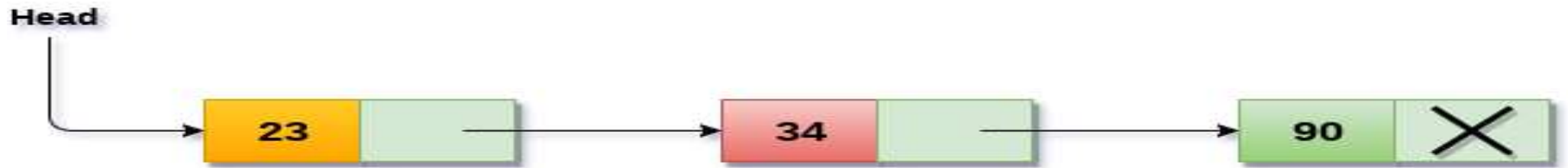
- ❖ Singly linked list can be defined as the collection of ordered set of elements/nodes.
- ❖ Singly linked list can be traversed only in one direction, as each node has only one link part
- ❖ Each link part contains address of the next node, hence list can not be traversed in reverse direction.
- ❖ Link part of the last node contains NULL value indicating the end of the list.



Types of lists

Singly Linked list

Example - Marks obtained by the student in three subjects are stored in a linked list.

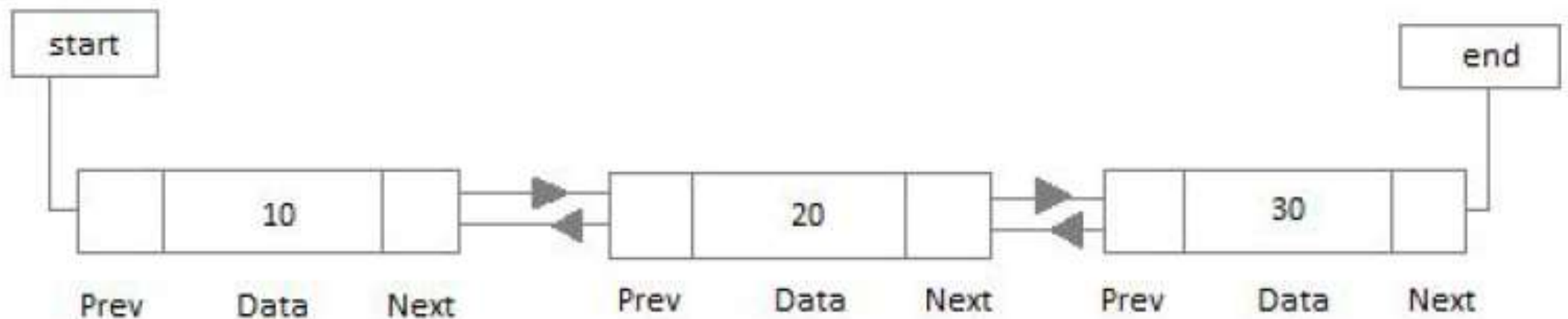


- The arrow represents the links.
- The data part of each node contains the marks obtained by the student in the different subjects.
- The last node in the list is identified by the null pointer in the address part of the last node.

Types of lists

Doubly linked list

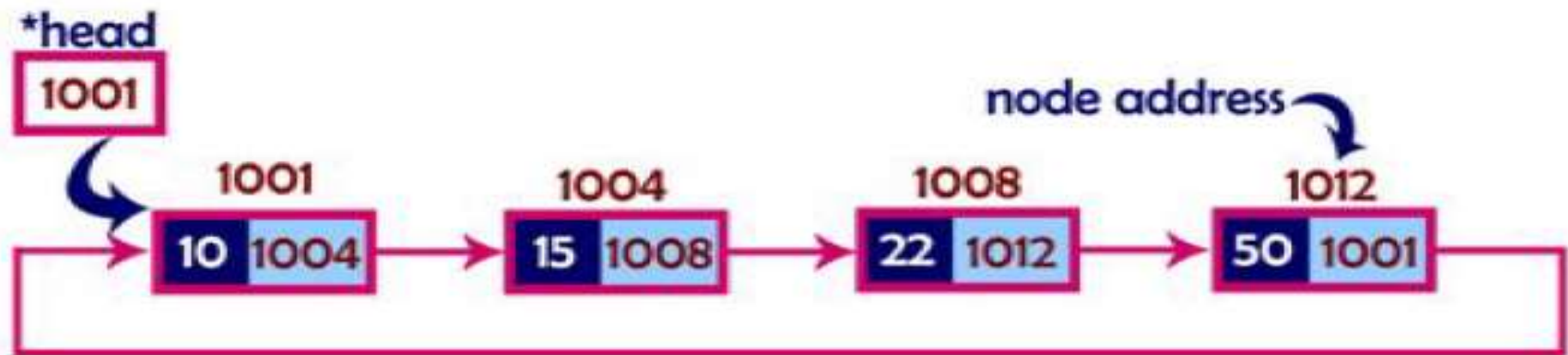
- ❖ Each node contains three fields
- ❖ One is data part which contain **data** only.
- ❖ Two other fields are links/addresses, one pointing to the **previous** node and another to the **next** node in the sequence of nodes.
- ❖ The beginning and ending nodes' previous and next links, respectively, point to some kind of terminator.

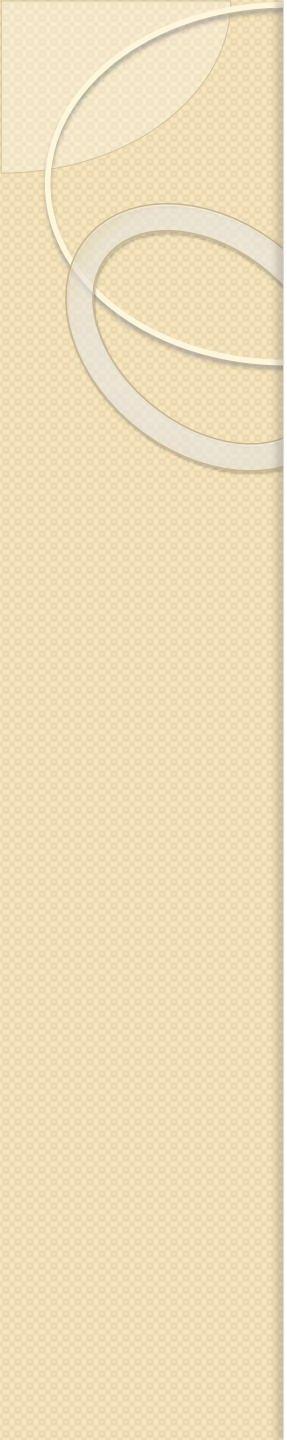


Types of lists

Circular Linked List

- ❖ In single linked list, every node points to its next node in the sequence and the last node points to NULL.
- ❖ Circular linked list is a sequence of elements in which every element has link to its next element and the last element has a link to the first element in the list.
- ❖ Circular linked list is similar to the single linked list except that the last node points to the first node in the list.





Operations on Single Linked Lists

1. Traversing
2. Creation
3. Insertion
4. Deletion
5. Searching
6. Concatenation
7. Display

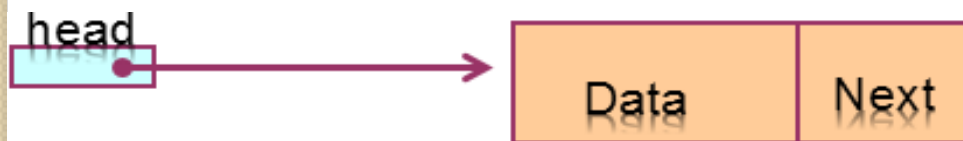
Creation

In a single linked list, the **address** of first node is stored in a reference node known as "**head**"/"**front**".

The **next** part (reference part) of the last node must be **NULL** always.

A node is represented as-
struct node

```
{    int data;  
    struct node *next; };
```



Algorithm to create a node

Nptr Createnode()

Begin

1. Allocate memory for header node.
H=(nptr)(malloc(sizeof(struct node)));
2. Verify the memory allocation.
If(H==NULL) then
 Printf("Memory not Allocated")
 Return;
3. Allocate next of header as NULL.
H → next=NULL;
4. Return H.

Creation

- ❖ **If there are n number of nodes in the initial linked list:**
 - ❖ Allocate **n** records, one by one.
 - ❖ Read in the fields of the records.
 - ❖ Modify the links of the records so that the chain is formed.

Algorithm Createlist()

Begin

1. Create empty list.

h=(nptr)(malloc(sizeof(struct node)));

h → next=NULL; Temp=h;

2. Repeat following steps while user enters positive choice.

Read x;

3. Allocate the space for new node.

new=(nptr)(malloc(sizeof(struct node)));

4. Assign x value to new node data and link to NULL.

new->data=x; new->next=temp->next;

5. Link new node with previous node

temp->next=new; temp=new;

Traversing the List

Once linked list has been constructed and head points to the first node-

- ❖ Follow the pointers.
- ❖ Display the contents of the nodes as they are traversed.
- ❖ Stop when the next pointer points to NULL.

Algorithm

STEP 1: SET PTR = HEAD/ START

STEP 2: IF PTR = NULL

 WRITE "EMPTY LIST"

 GOTO STEP 7

 END OF IF

STEP 4: REPEAT STEP 5 AND 6 UNTIL PTR != NULL

STEP 5: PRINT PTR → DATA

STEP 6: PTR = PTR → NEXT/ LINK[PTR]

[END OF LOOP]

STEP 7: EXIT

Inserting a Node in a List

Algorithm to setup

First setup empty list before implementing actual operations.

Step 1: Include all header files to be used in the program.

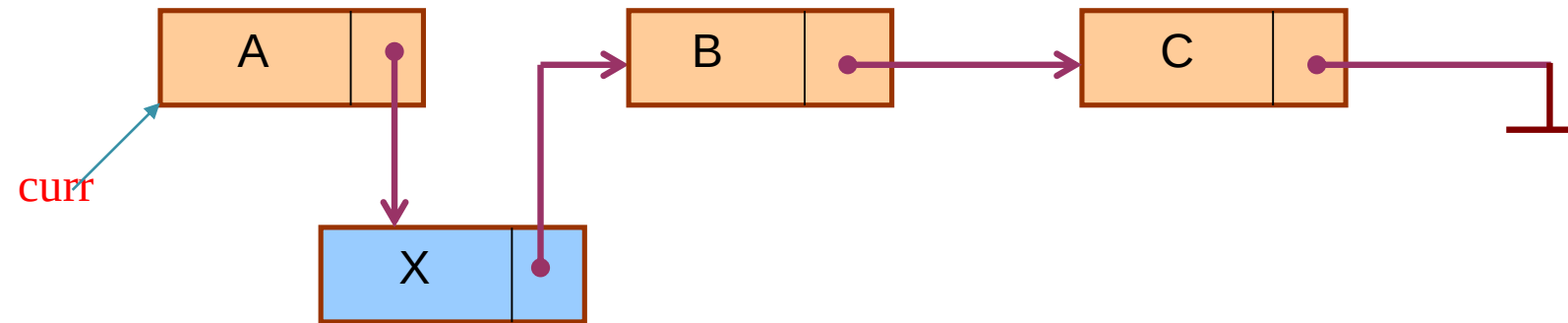
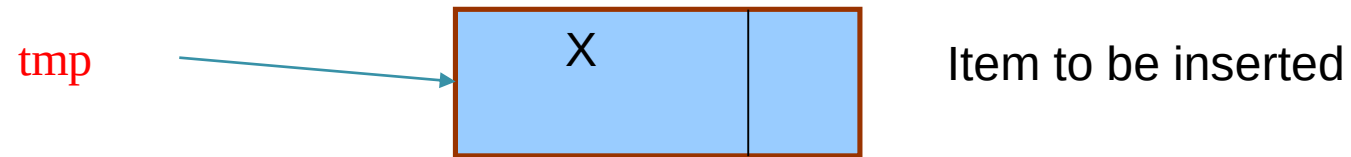
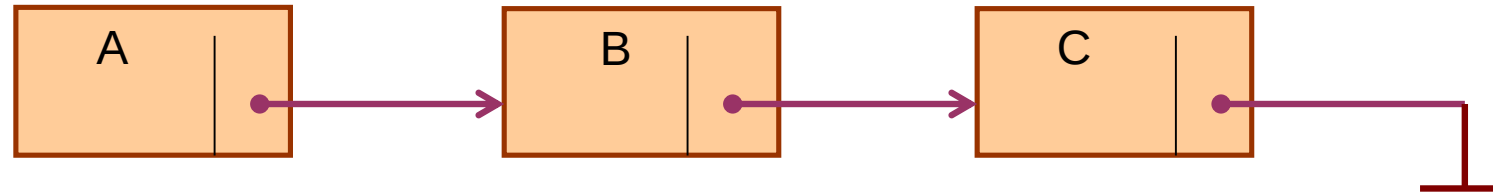
Step 2: Declare all user defined functions.

Step 3: Define a Node structure with two members data and next

Step 4: Define a Node pointer '**head**' and set it to **NULL**.

Step 5: Implement the main method by displaying operations menu and make suitable function calls in the main method to perform user selected operation.

Inserting a Node in a List



Insertion

In a single linked list, insertion operation can be performed in three ways-

i. Inserting at Beginning of the list

- ❖ Only one next pointer needs to be modified.
- ❖ *head* is made to point to the new node.
- ❖ New node points to the previously first element.

ii. Inserting at End of the list

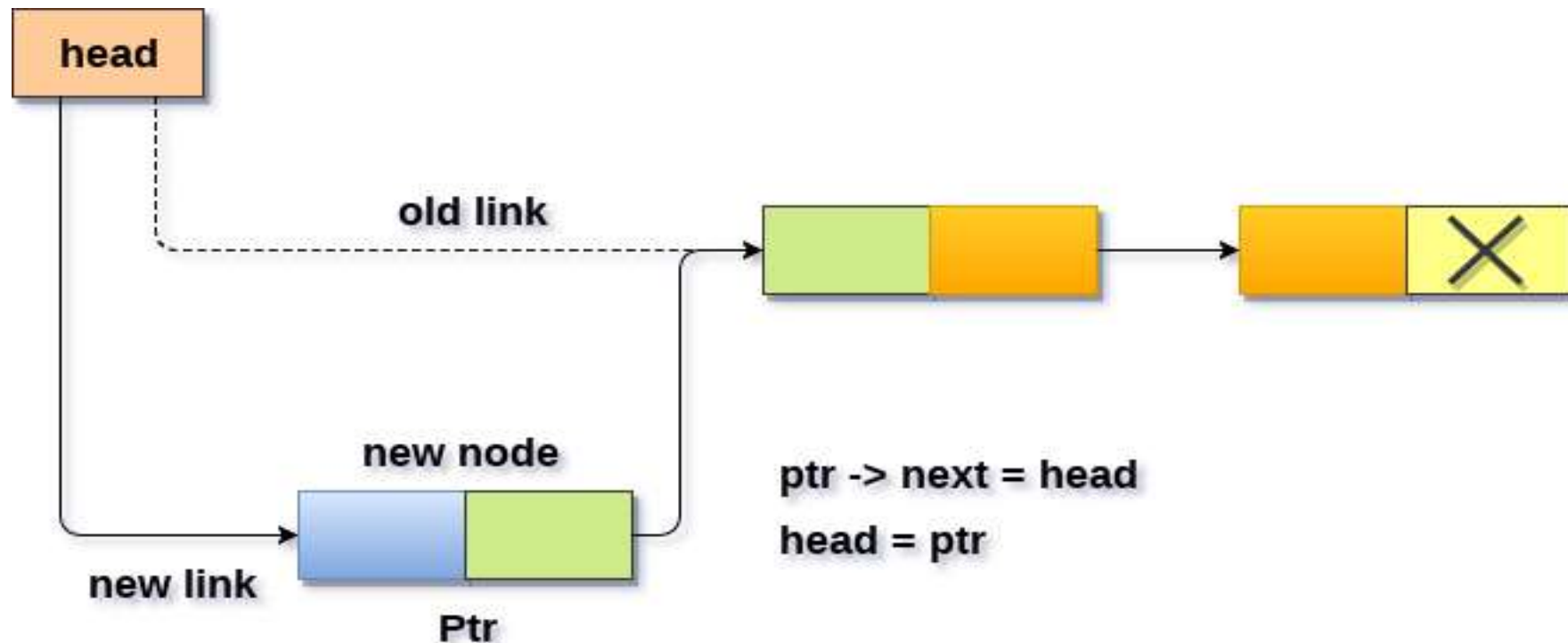
- ❖ Two next pointers need to be modified.
- ❖ Last node now points to the new node.
- ❖ New node points to NULL.

iii. Inserting at Specific location in the list

- ❖ Two next pointers need to be modified.
- ❖ Previous node now points to the new node.

i. Insertion at Beginning of the list

- ❖ Create a new node
- ❖ Next part of new node points to head of linked list.
- ❖ Change the head to the newly added node.



i. Insertion at Beginning of the list

Algorithm

Step 1: IF PTR = NULL

Write OVERFLOW AND Go to Step 7

[END OF IF]

Step 2: SET NEW_NODE = PTR

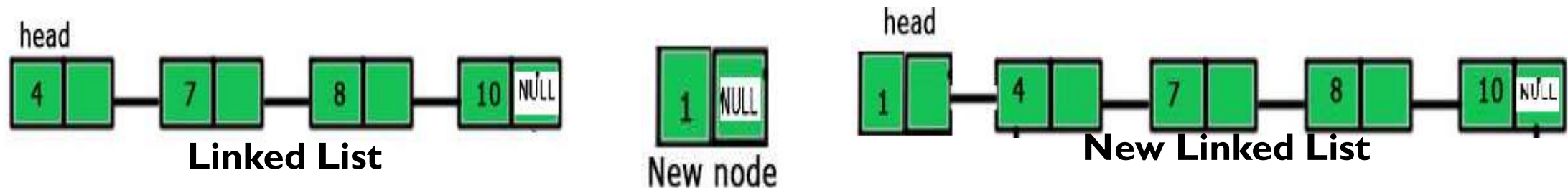
Step 3: SET PTR = PTR → NEXT

Step 4: SET NEW_NODE → DATA = VAL

Step 5: SET NEW_NODE → NEXT = HEAD

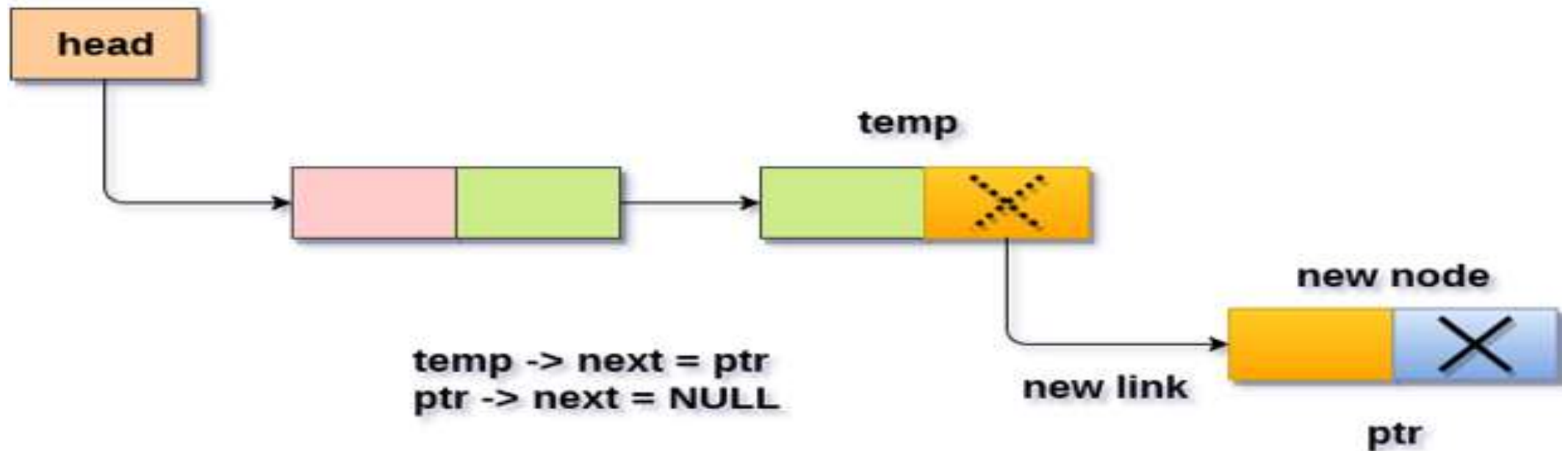
Step 6: SET HEAD = NEW_NODE

Step 7: EXIT



ii. Insertion at End of the list

- ❖ Traverse the Linked list till the end.
- ❖ Add a new node at the end of the list
- ❖ Change next field of last node to point to new node.



ii. Insertion at End of the list

Algorithm

Step 1: IF PTR = NULL

Write OVERFLOW ; Go to Step 10

Step 2: SET NEW_NODE = PTR

Step 3: SET PTR = PTR - > NEXT

Step 4: SET NEW_NODE - > DATA = VAL

Step 5: SET NEW_NODE - > NEXT = NULL

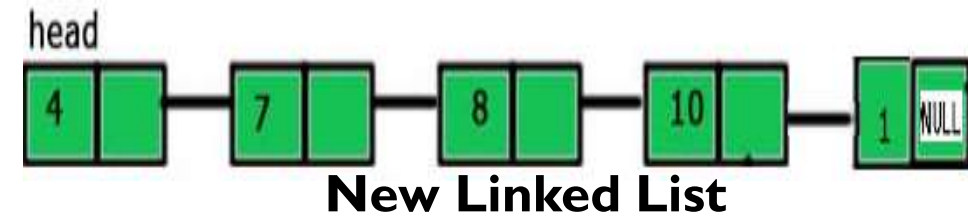
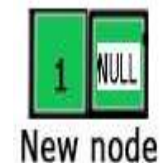
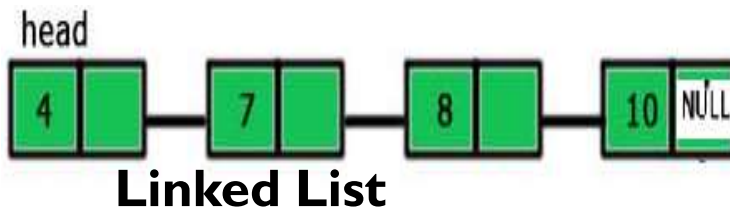
Step 6: SET PTR = HEAD

Step 7: Repeat Step 8 while PTR - > NEXT != NULL

Step 8: SET PTR = PTR - > NEXT

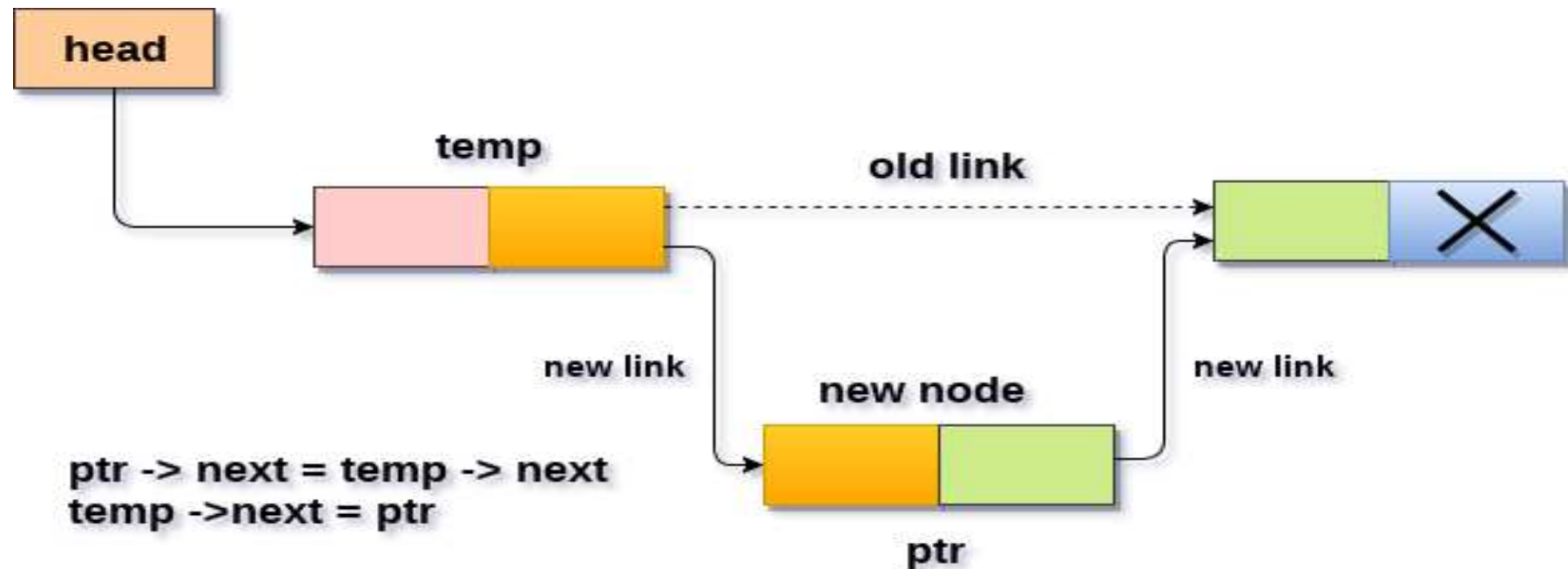
Step 9: SET PTR - > NEXT = NEW_NODE

Step 10: EXIT



iii. Insertion at specific location of the list

- ❖ Consider two nodes a & b.
- ❖ Make a new node.
- ❖ Point the 'next' of 'a' to the new node.
- ❖ Point the 'next' of the new node to the node 'b'.



iii. Insertion at specific location of the list

Algorithm

STEP 1: IF PTR = NULL

WRITE OVERFLOW and GOTO STEP 12

STEP 2: SET NEW_NODE=PTR, NEW_NODE→DATA=VAL

STEP 3: SET TEMP = HEAD,

STEP 4: SET I = 0

STEP 5: REPEAT STEPS 5-9 UNTIL I<loc

STEP 6: TEMP = TEMP → NEXT

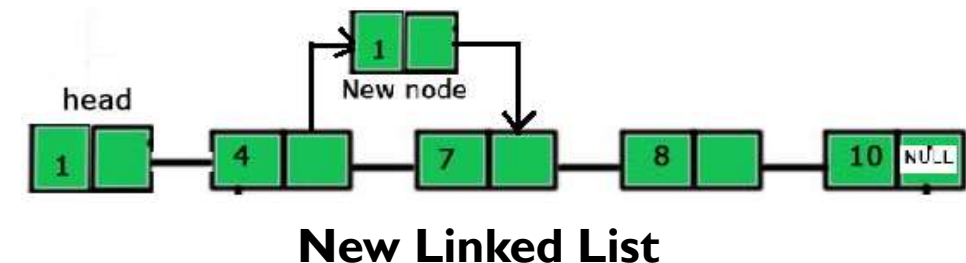
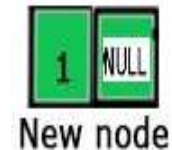
STEP 7: IF TEMP = NULL

WRITE " NODE NOT PRESENT"; GOTO STEP 10

STEP 8: PTR→NEXT = TEMP→NEXT ; TEMP→NEXT = PTR

STEP 9: SET PTR = NEW_NODE

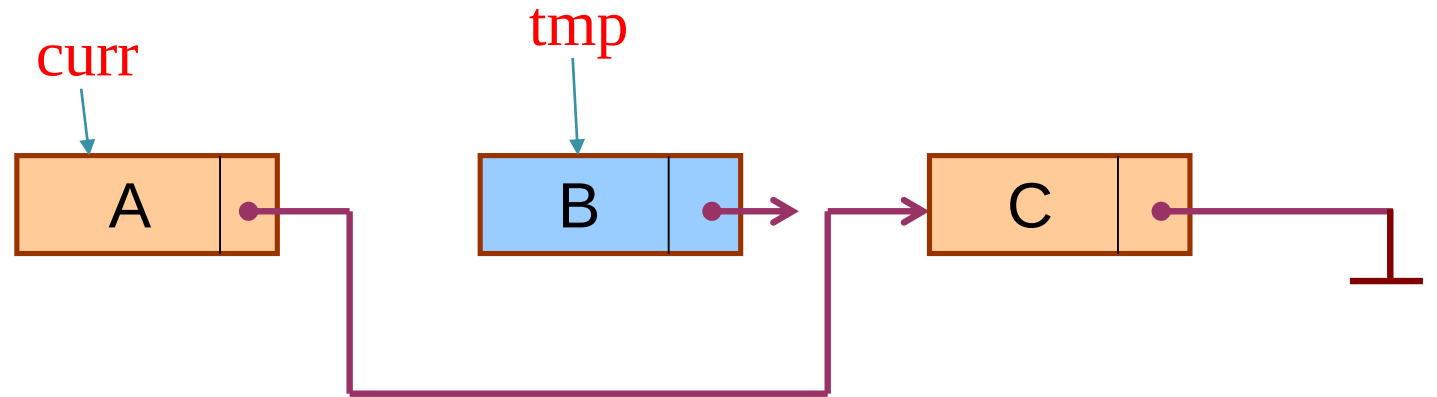
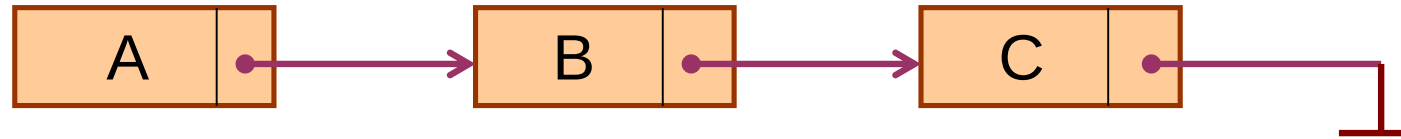
STEP 10: EXIT



Deletion

```
void delete(node *curr)
{
    node * tmp;
    tmp=curr->next;
    curr->next=tmp->next;
    free(tmp);
}
```

Item to be deleted



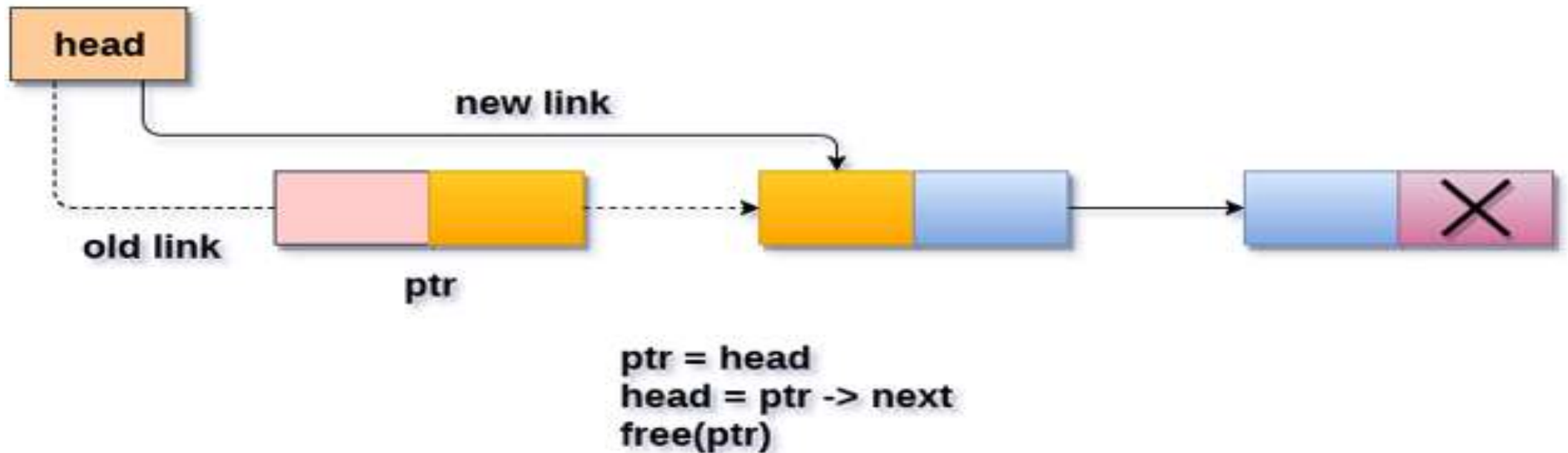
Deletion

Deletion operation can be performed in single linked list in three ways:-

- i. Deleting from Beginning of the list
- ii. Deleting from End of the list
- iii. Deleting a Specific Node

i. Deletion from Beginning of the list

- ❖ Make the head, point to the next of the head
- ❖ Free the pointer which was pointing to head node of the list



i. Deletion from Beginning of the list

Algorithm

Step 1: IF HEAD = NULL

Write UNDERFLOW

Go to Step 5

[END OF IF]

Step 2: SET PTR = HEAD / START

Step 3: SET HEAD = HEAD -> NEXT

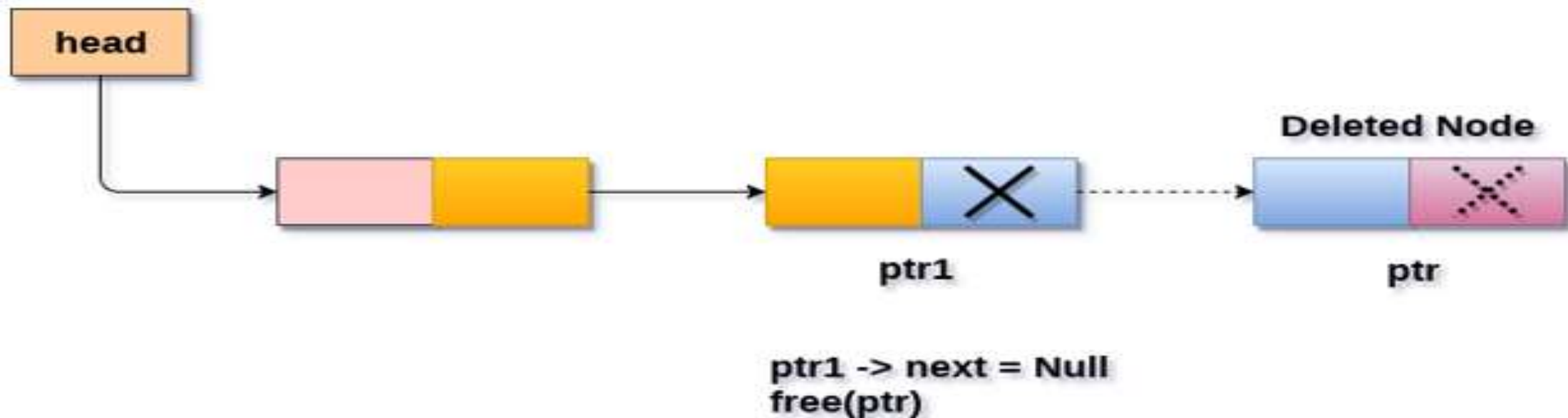
Step 4: FREE PTR

Step 5: EXIT

ii. Deletion from End of the list

Two scenarios in which a node is deleted from end-

1. There is only one node in list
2. There are more than one node in the list



ii. Deletion from End of the list

Algorithm

Step 1: IF START = NULL

 Write UNDERFLOW;

 Go to Step 8

Step 2: SET PTR = START

Step 3: Repeat Steps 4 and 5 while PTR -> NEXT!= NULL

Step 4: SET PREPTR = PTR

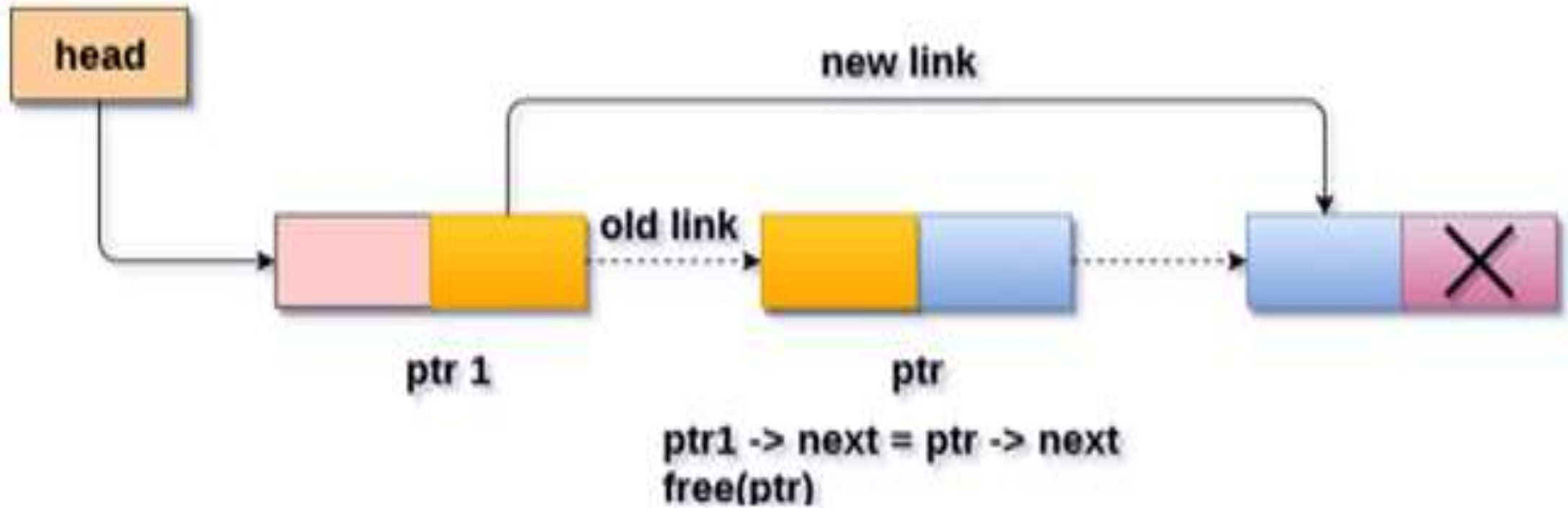
Step 5: SET PTR = PTR -> NEXT

Step 6: SET PREPTR -> NEXT = NULL

Step 7: FREE PTR

Step 8: EXIT

iii. Deletion of a Specific Node



iii. Deletion of a Specific Node

Algorithm

STEP 1: IF HEAD = NULL

WRITE UNDERFLOW, GOTO STEP 10

STEP 2: SET TEMP = HEAD

STEP 3: SET I = 0

STEP 4: REPEAT STEP 5 TO 8 UNTIL I < loc

STEP 5: TEMP I = TEMP

STEP 6: TEMP = TEMP → NEXT

STEP 7: IF TEMP = NULL Then WRITE "NODE NOT PRESENT", GO TO STEP 12

STEP 8: I = I + 1

END OF LOOP

STEP 9: TEMP I → NEXT = TEMP → NEXT

STEP 10: FREE TEMP

STEP 11: EXIT

Searching in singly linked list

Algorithm

Step 1: SET PTR = HEAD / START

Step 2: Set I = 0

STEP 3: IF PTR = NULL

WRITE "EMPTY LIST", GOTO STEP 8

STEP 4: REPEAT STEP 5 TO 7 UNTIL PTR != NULL

STEP 5: if ptr $\rightarrow$ data = item

SET LOC = PTR

Write i+1

End of IF

STEP 6: I = I + 1

STEP 7: PTR = PTR $\rightarrow$ NEXT / LINK [PTR]

STEP 8: EXIT

Displaying a Single Linked List

Algorithm

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty then, display 'List is Empty' and terminate the function.

Step 3: If it is Not Empty then, define a Node pointer 'temp' and initialize with head.

Step 4: Keep displaying temp → data with an arrow (-->) until temp reaches to the last node

Step 5: Display temp → data with arrow pointing to NULL
(temp → data --> NULL).

THANK YOU